

Beginner's Guide: Building the Crime Analytics Prototype on Zoho Catalyst

Pin-to-pin steps for a working MVP — not the full-fledged platform

This is the **cut-down, buildable-in-a-hackathon** version of the full blueprint. It skips the AI chatbot, network graph, and ML scoring for now, and gets you a real, working, end-to-end app: login → search cases → see repeat offenders → see a dashboard. Once this works, the full blueprint's advanced features slot in on top of it.

Commands and console click-paths below were verified against Zoho Catalyst's official documentation (docs.catalyst.zoho.com) as of July 2026. Catalyst's console UI changes over time — if a button name doesn't match exactly, look for the nearest equivalent.

0. What "prototype" means here (scope note)

Building all 25 tables + full NLP chatbot + graph algorithms first is how hackathon teams run out of time. Instead, we implement a **minimal vertical slice**: 7 of the 25 tables (enough to demonstrate the real investigative value — case search, repeat offenders, dashboard stats), one working REST API, one simple web page, deployed and reachable by a URL. Everything else from the full blueprint is added the same way, table by table, function by function.

Included in this prototype	Deferred to later (see full blueprint)
7 core tables (District, Unit, CrimeHead, CaseMaster, Accused, Victim, ArrestSurrender)	Remaining 18 tables (Act, Section, Court, Employee, etc.)
3 working API endpoints (search, repeat offenders, dashboard stats)	Full NL → SQL AI chatbot
Basic login (Catalyst Authentication)	Full 5-role RBAC
One HTML page: dashboard + search + a "canned questions" dropdown	Voice, Kannada, network graph, ML scoring

1. Prerequisites

Tool	Why you need it	How to check
A Zoho account	To create your Catalyst project (project creation for your first project must be done from the web console, not CLI)	Sign up free at catalyst.zoho.com
Node.js (v14+) and npm	Catalyst CLI and your Functions both run on Node	<code>node -v</code> in terminal
A code editor	VS Code recommended — Zoho even publishes a "Catalyst	—

Tool	Why you need it	How to check
	Tools" extension	
Terminal / command prompt	You'll run all commands below from here	—

2. Step 1 — Create your Catalyst project (console)

Why: Catalyst requires your *first* project to be created via the web console; every service (Data Store, Functions, Auth, Web Hosting) is provisioned inside that one project.

1. Go to `catalyst.zoho.com` → sign in → **Create Project**.
2. Name it, e.g. `crime-analytics-prototype`.
3. Once created, you'll land on the project's console home — this is where Data Store, Functions, Web Hosting, and Authentication all live.

3. Step 2 — Install and initialize the Catalyst CLI

Why: The CLI is how you scaffold Functions, test locally, and deploy — instead of hand-editing everything in the browser.

```
# Install the CLI globally
npm install -g zcatalyst-cli

# Create a project folder and move into it
mkdir crime-analytics-prototype
cd crime-analytics-prototype

# Initialize Catalyst in this folder
catalyst init
```

`catalyst init` will prompt you to:

- Log in (opens a browser window) if you aren't already.
- **Select the project** you created in Step 1 from the list, and associate this folder with it.
- **Select components to initialize** — choose **Functions** and **Client** (this is the Web Hosting piece). You don't need to initialize Data Store or Authentication here; those are configured from the console (Step 4) and used from your Function code via the SDK.

You'll get a folder structure roughly like:

```
crime-analytics-prototype/
├── functions/
├── client/
└── catalyst.json    ← records which components/resources are linked to this project
```

4. Step 3 — Design the minimal prototype schema

Why only 7 tables: This subset is enough to answer the headline demo questions — "show cases in district X," "who are the repeat offenders," "what's today's case count" — while staying faithful to the real primary/foreign keys from the uploaded ER diagram (no invented columns).

```
erDiagram
    District ||--o{ Unit : "has"
    Unit ||--o{ CaseMaster : "registers"
    CrimeHead ||--o{ CaseMaster : "classifies"
    CaseMaster ||--o{ Accused : "names"
    CaseMaster ||--o{ Victim : "involves"
    Accused ||--o{ ArrestSurrender : "linked to"
    CaseMaster ||--o{ ArrestSurrender : "has"

    District {
        bigint ROWID PK
        string DistrictName
    }
    Unit {
        bigint ROWID PK
        string UnitName
        bigint DistrictID FK
    }
    CrimeHead {
        bigint ROWID PK
        string CrimeGroupName
    }
    CaseMaster {
        bigint ROWID PK
        string CrimeNo
        date CrimeRegisteredDate
        bigint PoliceStationID FK
        bigint CrimeMajorHeadID FK
        decimal latitude
        decimal longitude
        string CaseStatus
        text BriefFacts
    }
    Accused {
        bigint ROWID PK
        string AccusedName
        bigint AgeYear
        string GenderID
        bigint CaseMasterID FK
    }
    Victim {
        bigint ROWID PK
        string VictimName
        bigint AgeYear
        string GenderID
        bigint CaseMasterID FK
    }
    ArrestSurrender {
        bigint ROWID PK
        bigint AccusedMasterID FK
        bigint CaseMasterID FK
        date ArrestSurrenderDate
    }
```

Key beginner fact about Catalyst Data Store: every table automatically gets a ROWID column that acts as its primary key — you never define your own PK column. When one table needs to

reference another (a foreign key), you just add a `Number/BigInt` column and store the parent row's `ROWID` in it. That's what `DistrictID`, `PoliceStationID`, `CaseMasterID`, etc. are doing above.

5. Step 4 — Create the tables in Data Store (console)

Why Data Store, not a separate MySQL/Mongo box: it's already inside your project, it's schema-flexible enough for a hackathon pace, and you query it with **ZCQL**, a SQL-like language — so everything in the full blueprint's SQL library ports over almost unchanged.

For **each** of the 7 tables, repeat this exact click-path:

1. In the console, go to **Cloud Scale** → **Data Store** (under Storage).
2. Click **Create a new Table**.
3. Type the table name exactly as below (no spaces, must start with a letter) → **Create**.
4. In the table's **Schema View**, click **New Column** for every column in the table below → choose the **Data Type** → set options → **Create**.

Table	Column	Data Type	Options
District	DistrictName	Var Char (max length 100)	Is Mandatory ✓
Unit	UnitName	Var Char (100)	Is Mandatory ✓
	DistrictID	BigInt	—
CrimeHead	CrimeGroupName	Var Char (150)	Is Mandatory ✓
CaseMaster	CrimeNo	Var Char (30)	Is Unique ✓, Search Index ✓
	CrimeRegisteredDate	Date	Is Mandatory ✓
	PoliceStationID	BigInt	—
	CrimeMajorHeadID	BigInt	—
	latitude	Decimal	—
	longitude	Decimal	—
	CaseStatus	Var Char (30)	e.g. "Under Investigation", "Closed"
	BriefFacts	Text	—
Accused	AccusedName	Var Char (100)	Search Index ✓
	AgeYear	BigInt	—
	GenderID	Var Char (1)	e.g. M/F/T
	CaseMasterID	BigInt	—
Victim	VictimName	Var Char (100)	—

Table	Column	Data Type	Options
	AgeYear	BigInt	—
	GenderID	Var Char (1)	—
	CaseMasterID	BigInt	—
ArrestSurrender	AccusedMasterID	BigInt	—
	CaseMasterID	BigInt	—
	ArrestSurrenderDate	Date	—

Tip: turn on **Search Index** for **CrimeNo** and **AccusedName** now — it's what powers Catalyst's Data Store Search / your quick-search box later, and it's a single toggle at creation time.

6. Step 5 — Seed sample data

Why: you need a few realistic rows to demo against before wiring up the AI layer.

The fastest beginner path is the console itself:

1. Open a table → **Data View** tab → + **New Row** → fill in values → **Save**.
2. Add: 2 districts, 2–3 units (pointing **DistrictID** at a real District ROWID), 3–4 crime heads, 6–8 cases (pointing **PoliceStationID/CrimeMajorHeadID** at real ROWIDs), a handful of accused (some **repeating the same AccusedName across 2–3 different CaseMasterIDs** — this is what makes your "repeat offenders" query return something interesting), a few victims, a couple of arrests.

Once you're comfortable, you can seed faster with a small Node script using the SDK (same call your Functions will use):

```
const catalyst = require('zcatalyst-sdk-node');
// run inside a Catalyst function, or via `catalyst functions:shell` for local testing
module.exports = async (req, res) => {
  const app = catalyst.initialize(req);
  const table = app.datastore().table('CaseMaster');
  await table.insertRow({
    CrimeNo: '104430006202600001',
    CrimeRegisteredDate: '2026-01-15',
    PoliceStationID: 1,           // ROWID of a Unit row you created
    CrimeMajorHeadID: 1,         // ROWID of a CrimeHead row you created
    latitude: 12.9716,
    longitude: 77.5946,
    CaseStatus: 'Under Investigation',
    BriefFacts: 'Sample seeded case'
  });
  res.send('seeded');
};
```

7. Step 6 — Write your first Catalyst Function (the case search API)

Why Functions: this is your serverless backend — no server to provision, and it talks to Data Store via the same SDK.

```
# From your project folder
catalyst function:add
```

When prompted:

- **Function type** → Advanced I/O (gives you full request/response control — best for a REST API).
- **Stack** → Node.js.
- **Function reference name** → caseSearch.
- Fill in package name / entry file / author email as prompted.

This creates `functions/caseSearch/`. Open `functions/caseSearch/index.js` and replace its contents:

```
const catalyst = require('zcatalyst-sdk-node');

module.exports = async (req, res) => {
  const app = catalyst.initialize(req);
  const zcql = app.zcql();

  const { district, from, to } = req.query;

  // Build a ZCQL query — this is SQL-like, so it maps directly to the
  // 100-query library in the full blueprint.
  let query = `SELECT CaseMaster.ROWID, CaseMaster.CrimeNo,
CaseMaster.CrimeRegisteredDate,
                CaseMaster.CaseStatus, Unit.UnitName,
District.DistrictName
                FROM CaseMaster
                JOIN Unit ON CaseMaster.PoliceStationID = Unit.ROWID
                JOIN District ON Unit.DistrictID = District.ROWID`;

  const conditions = [];
  if (district) conditions.push(`District.DistrictName = '${district}'`);
  if (from) conditions.push(`CaseMaster.CrimeRegisteredDate >= '${from}'`);
  if (to) conditions.push(`CaseMaster.CrimeRegisteredDate <= '${to}'`);
  if (conditions.length) query += ' WHERE ' + conditions.join(' AND ');

  try {
    const result = await zcql.executeZCQLQuery(query);
    res.status(200).send({ count: result.length, results: result });
  } catch (err) {
    res.status(500).send({ error: err.message });
  }
};
```

Beginner note on SQL-injection safety: the example above builds the WHERE clause with string concatenation to keep it readable. Before you show this to real users, switch to ZCQL's parameterised query builder (`zcql.selectQuery(...)`) instead of concatenating strings — worth doing even for a hackathon demo if you have 15 minutes spare.

8. Step 7 — Add the repeat-offenders and dashboard endpoints

catalyst function:add

Repeat the same prompts, name it `analytics`. Then in `functions/analytics/index.js`:

```
const catalyst = require('zcatalyst-sdk-node');

module.exports = async (req, res) => {
  const app = catalyst.initialize(req);
  const zcql = app.zcql();
  const action = req.query.action;

  try {
    if (action === 'repeat-offenders') {
      const query = `SELECT Accused.AccusedName, COUNT(Accused.CaseMasterID) AS
CaseCount
                    FROM Accused
                    GROUP BY Accused.AccusedName
                    HAVING COUNT(Accused.CaseMasterID) > 1`;
      const result = await zcql.executeZCQLQuery(query);
      return res.status(200).send(result);
    }

    if (action === 'dashboard') {
      const today = new Date().toISOString().slice(0, 10);
      const todayCount = await zcql.executeZCQLQuery(
        `SELECT COUNT(CaseMaster.ROWID) AS c FROM CaseMaster WHERE
CaseMaster.CrimeRegisteredDate = '${today}'`
      );
      const activeCount = await zcql.executeZCQLQuery(
        `SELECT COUNT(CaseMaster.ROWID) AS c FROM CaseMaster WHERE
CaseMaster.CaseStatus != 'Closed'`
      );
      return res.status(200).send({ todayCases: todayCount, activeCases:
activeCount });
    }

    res.status(400).send({ error: 'Unknown action. Use ?action=repeat-offenders
or ?action=dashboard' });
  } catch (err) {
    res.status(500).send({ error: err.message });
  }
};
```

Why two Functions instead of one: keeping "search" and "analytics" separate mirrors the full blueprint's backend/`functions/cases/` vs backend/`functions/analytics/` split, so migrating to the full architecture later is just adding folders, not rewriting.

9. Step 8 — Test locally

catalyst serve

This starts a local server and prints URLs like:

caseSearch → <http://localhost:3000/server/caseSearch>

analytics → http://localhost:3000/server/analytics

Test in your browser or with curl:

```
curl "http://localhost:3000/server/caseSearch?district=Mysuru"
curl "http://localhost:3000/server/analytics?action=repeat-offenders"
curl "http://localhost:3000/server/analytics?action=dashboard"
```

If you get JSON back with your seeded rows, your backend works end-to-end.

10. Step 9 — Build the minimal frontend

Why plain HTML/JS for the prototype: React/Tailwind polish (from the full blueprint) can come later — a single static page is faster to wire up and deploy for a first demo.

In `client/`, create `index.html`:

```
<!DOCTYPE html>
<html>
<head>
  <title>Crime Analytics – Prototype</title>
  <style>
    body { font-family: sans-serif; max-width: 800px; margin: 40px auto; }
    table { width: 100%; border-collapse: collapse; margin-top: 16px; }
    td, th { border: 1px solid #ccc; padding: 6px 10px; text-align: left; }
    .card { display: inline-block; padding: 16px 24px; border: 1px solid #ccc;
border-radius: 8px; margin-right: 12px; }
  </style>
</head>
<body>
  <h1>Crime Analytics Prototype</h1>

  <div id="dashboard"></div>

  <h2>Quick questions</h2>
  <select id="quickQuestions">
    <option value="">Choose a question...</option>
    <option value="search">Show all cases</option>
    <option value="repeat">Find repeat offenders</option>
  </select>
  <button onclick="runQuestion()">Ask</button>

  <div id="results"></div>

  <script>
    const BASE = ''; // filled after deploy – leave blank while testing with
`catalyst serve`

    async function loadDashboard() {
      const res = await fetch(`${BASE}/server/analytics?action=dashboard`);
      const data = await res.json();
      document.getElementById('dashboard').innerHTML = `
        <div class="card"><strong>${data.todayCases[0].c}</strong><br>Today's
cases</div>
        <div class="card"><strong>${data.activeCases[0].c}</strong><br>Active
cases</div>`;
    }

    async function runQuestion() {
      const q = document.getElementById('quickQuestions').value;
```

```

    let rows = [];
    if (q === 'search') {
      const res = await fetch(`${BASE}/server/caseSearch`);
      rows = (await res.json()).results;
      renderTable(rows, ['CrimeNo', 'CrimeRegisteredDate', 'CaseStatus',
'UnitName', 'DistrictName']);
    } else if (q === 'repeat') {
      const res = await fetch(`${BASE}/server/analytics?action=repeat-
offenders`);
      rows = await res.json();
      renderTable(rows, ['AccusedName', 'CaseCount']);
    }
  }

  function renderTable(rows, cols) {
    let html = '<table><tr>' + cols.map(c => `<th>${c}</th>`).join('') +
'</tr>';
    rows.forEach(r => {
      html += '<tr>' + cols.map(c => `<td>${r[c] ?? ''}</td>`).join('') +
'</tr>';
    });
    html += '</table>';
    document.getElementById('results').innerHTML = html;
  }

  loadDashboard();
</script>
</body>
</html>

```

This dropdown is deliberately a "**chatbot-lite**" — canned questions mapped to real endpoints. It demonstrates the same user-facing idea as the full NL → SQL chatbot without needing the LLM integration yet; swap it for the real chatbot Function once the base app is stable.

11. Step 10 — Deploy

Why Web Hosting for the client: it's static hosting inside the same Catalyst project — one deploy command, no separate hosting account.

```
catalyst deploy
```

This deploys both your Functions and your client together (based on what's initialized in `catalyst.json`). At the end, the CLI prints your live URLs, e.g.:

```

Client: https://crime-analytics-prototype-
<id>.development.catalystserverless.in
Function caseSearch: https://crime-analytics-prototype-
<id>.development.catalystserverless.in/server/caseSearch

```

Update the `BASE` variable in `index.html` if you deployed the client to a different domain than the functions (usually not necessary — they share the same project domain), then redeploy.

12. Step 11 (optional stretch) — Basic login with Catalyst Authentication

Why: the full blueprint's RBAC depends on this; adding a bare-bones version now (one login page, one "role" field) means Step 12 in the full plan is a small extension, not a rebuild.

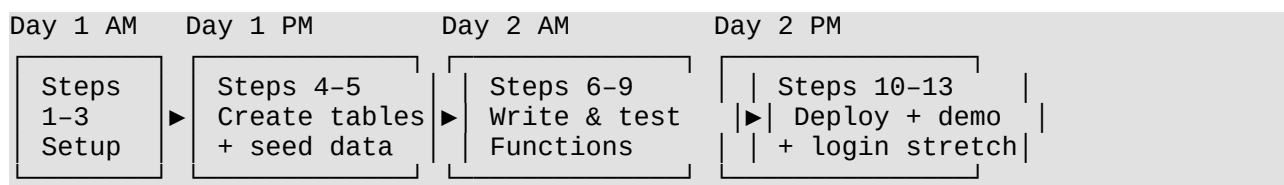
1. Console → **Authentication** → enable it for your project, add a test user (e.g. an Investigator).
2. Add a `Role` field to the Catalyst Authentication user profile (Var Char), set to `Investigator` for your test user.
3. In each Function, read the logged-in user from the request (`catalyst.initialize(req)` already gives you the authenticated user context) and, before running any query, check their `Role` — for the prototype this can be as simple as an `if (role !== 'Investigator') return res.status(403).send(...)` at the top of the Function.

Keep this to one role for the prototype demo; the full 5-role matrix (§4 of the main blueprint) is a config/data change, not new code.

13. End-to-end test checklist

- ☐ `catalyst serve` runs locally with no errors
 - ☐ `curl` against `caseSearch` and `analytics` return your seeded rows
 - ☐ Dashboard cards on the page show real counts
 - ☐ "Show all cases" and "Find repeat offenders" both render tables
 - ☐ `catalyst deploy` succeeds and the public URL loads the page
 - ☐ The deployed page's dashboard and quick questions work against the live URL (not just localhost)
-

Prototype build order (visual)



Where this plugs into the full blueprint

Once the checklist above is green, growing this into the full platform is additive:

- More tables → repeat Step 4's click-path for the remaining 18 tables in the ER diagram.
- Real chatbot → replace the <select> dropdown with a chat box calling a new chatbot Function that does intent detection + ZCQL generation (§19 of the full blueprint) instead of the two hardcoded actions above.
- Network graph / hotspot clustering → add an AppSail job once you have enough seeded case volume to make clustering meaningful.
- Full RBAC, voice, Kannada, PDF reports, Cron/Signals → each is a new Function or console toggle added to this same project, not a new project.